Übung 9 - TypeScript 2 Lösung

Aufgabe 1 (**Funktionen in TypeScript**)

(a) Berechnen Sie die Fibonacci-Funktion mit einer rekursiven Funktion. Zusatzfrage: Wie oft wird die Funktion fib aufgerufen, um z.B. fib(5) zu berechnen?

Lösung:

```
1   {
2   function fib(n : number):number {
3       switch(n) {
4           case 0: return 1;
5           case 1: return 1;
6           default: return fib(n-1)+fib(n-2);
7       }
8   }
9   for (let i=2;i<10;i++)      console.log(fib(i));
10  }
```

(b) Berechnen Sie den größten gemeinsamen Teiler (ggT) mit einer rekursiven Funktion.

Lösung:

```
1  {   function ggt(a : number, b: number):number {
2          if (a===b) return a;
3          else if (a>b) return ggt(b, a-b);
4          else return ggt(a, b-a)
5      }
6
7      console.log(ggt(25,15) + " " + ggt(20,70));
8  }
```

(c) Schreiben Sie eine rekursive Funktion, die die Collatz-Folge überprüft. Diese Folge ist wie folgt definiert:
c(n)=n/2, falls n gerade
c(n)=3*n+1, falls n ungerade.
Behauptung: Die Folge endet immer mit 1.
Geben Sie jedes Folgenglied aus. Testen Sie mit 5, 17 und 23.

Lösung:

```
1   {
2   function c(n:number) : number {
3       console.log(n);
4       if (n===1) return 1;
5       else if (n%2===0) return c(n/2);
6       else return c(3*n+1);
7   }
8
```

```
    console.log(c(23));
}
```

(d) Auch in der Mathematik gibt es Lambda-Funktionen, wie z.B. $\lambda x.x+1$. D.h. die Funktion ohne Namen, die zu einem Argument eins hinzuaddiert. Definieren Sie die folgende Lambda-Funktion in Typescript: $\lambda x.\lambda y.x * y - x - y$.

Lösung:

```
{
// Schreiben Sie die Lambda-Funktion in Typescript:
// \x.\y.x*y-x-y
let lambdafkt : (x : number, y : number) => number =(x,y) =>
   { return x*y-x-y; }
console.log(lambdafkt(10,7));
}
```

(e) Wie wird der Mehrwertsteuersatz in der Funktion berechneMehrwertsteuer behandelt, wenn kein zweiter Parameter übergeben wird?

```
function berechneMehrwertsteuer(preis: number, satz: number =
   0.19): number {
   return preis * satz;
}

let nettopreis = 50;
let bruttopreis = nettopreis +
   berechneMehrwertsteuer(nettopreis);
```

Lösung: Wird kein zweiter Parameter übergeben, wird der Default 0.19 für `satz` angenommen.

(f) Schreiben Sie eine Funktion, welche eine unbestimmte Anzahl an Parametern (Zahlen) annehmen kann und die Summe der übergebenen Argumente zurückgibt. (Beispiel: sum(13, 21, 8) → 42 aber auch sum(1, 8, 2, 19, 30) → 60).

Lösung:

```
    { // Schreiben Sie eine Funktion, welche eine unbestimmte
       Anzahl an Parametern (Zahlen) bekommen kann und die
       Summe der übergebenen Parameter zurückgibt.
    // (Beispiel: sum(13, 21, 8) -> 42 aber auch sum(1, 8, 2,
       19, 30) -> 60).
    function sum(...numbers: number[]): number {
        let sum : number = 0;
        for (let i:number = 0; i < numbers.length; i++) sum
           += numbers[i];
        return sum;
        // Kürzere Lösung: return numbers.reduce((total,
           current) => total + current, 0);
    }

    console.log(sum(13, 21, 8));          // Ausgabe: 42
```

```
    console.log(sum(1, 8, 2, 19, 30));     // Ausgabe: 60
    }
```

Aufgabe 2 (**Klassen**)

Erstellen Sie eine Klasse `Book` in TypeScript, die folgende Eigenschaften und Methoden besitzt:

- Zwei Eigenschaften: `title: string` und `author: string`
- Einen Konstruktor, der beide Eigenschaften initialisiert
- Eine Methode `describe()`, die einen Text der Form `"Das Buch 'Der Titel' wurde von Der Autor geschrieben."` zurückgibt

Erzeugen Sie anschließend zwei Objekte dieser Klasse und geben Sie deren Beschreibung mit `console.log()` aus.

Lösung:

```
class Book {
  title: string;
  author: string;

  constructor(title: string, author: string) {
    this.title = title;
    this.author = author;
  }

  describe(): string {
    return `Das Buch '${this.title}' wurde von ${this.author}
        geschrieben.`;
  }
}

const book1 = new Book("Der Prozess", "Franz Kafka");
const book2 = new Book("Faust", "Johann Wolfgang von Goethe");

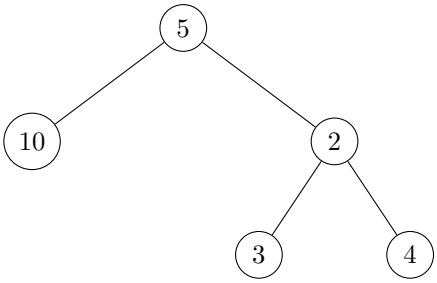
console.log(book1.describe());
console.log(book2.describe());
```

Aufgabe 3 (**Abstrakte Klassen**)

Modellieren Sie einen binären gerichteten azyklischen Graphen, d.h. einen binären Baum, indem Sie die folgende abstrakte Klasse `TreeNode` um zwei konkrete Klassen `LeafNode` (Blätter) und `InternalNode` (innere Knoten) erweitern. Die Funktion `calculateValue` gibt bei einem Blatt einen `value` zurück und bei inneren Knoten die Summe aus Value des linken und des rechten Teilbaums multipliziert mit dem Gewicht (`weight`) Wert des Knotens.

```
abstract class TreeNode {
    abstract calculateValue(): number;
}
```

Beispiel:



Der Wert dieses Baumes ist: $(10 + (3 + 4) * 2) * 5 = (10 + 14) * 5 = 120$.

Lösung:

```
class LeafNode extends TreeNode {
    private value: number;

    constructor(value: number) {
        super();
        this.value = value;
    }

    calculateValue(): number {
        return this.value;
    }
}

class InternalNode extends TreeNode {
    private weight: number;
    public leftChild: TreeNode;
    public rightChild: TreeNode;

    constructor(weight: number, leftChild: TreeNode, rightChild:
       TreeNode) {
        super();
        this.weight = weight;
        this.leftChild = leftChild;
        this.rightChild = rightChild;
    }

    calculateValue(): number {
        return (
            (this.leftChild.calculateValue() +
                this.rightChild.calculateValue()) *
            this.weight
        );
    }
}

const leftChild = new LeafNode(10);
const rightChild = new InternalNode(2, new LeafNode(3), new
   LeafNode(4));
const tree = new InternalNode(5, leftChild, rightChild);
console.log(tree.calculateValue());
```